

LAPORAN MACHINE LEARNING
Analisis Eksperimental Supervised Learning, Unsupervised Learning,
dan Prapemrosesan Data



Dibuat oleh:

Nama : Achmad Apanza

NPM : G1A024015

Kelas : A (Informatika)

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU
2026

BAB I PENDAHULUAN

1.1 Latar Belakang

Machine learning kini menjadi tulang punggung komputasi cerdas modern. Kemampuannya menarik inferensi langsung dari data empiris membuat sistem tidak lagi perlu diprogram melalui aturan eksplisit satu per satu (rule-based programming). Salah satu rujukan yang paling banyak dipakai untuk mempelajari bidang ini adalah buku *Introduction to Machine Learning with Python* karya Andreas C. Muller, anggota tim pengembang inti Scikit-Learn, bersama Sarah Guido. Di kalangan akademisi buku ini lebih dikenal sebagai 'buku ungu O'Reilly'. Isinya menguraikan dua paradigma besar secara sistematis: Supervised Learning (pembelajaran terawasi) dan Unsupervised Learning (pembelajaran tak terawasi).

Dalam konteks perkuliahan, memahami algoritma machine learning secara teoretis saja tidak cukup. Penguasaannya perlu diuji lewat implementasi kode, penelusuran kurva pembelajaran, analisis kompromi bias-varians, dan pengamatan langsung atas perilaku model ketika berhadapan dengan kondisi data nyata seperti perbedaan skala antarfitur dan struktur non-linear. Praktikum ini disusun untuk menutup jarak antara pemahaman konseptual dan pengalaman menjalankan eksperimen sendiri.

1.2 Tujuan Praktikum

Sasaran pembelajaran yang hendak dicapai melalui praktikum ini adalah sebagai berikut:

1. Mengimplementasikan dan menguji seluruh algoritma Supervised Learning pada Chapter 2, meliputi KNN, Regresi Linier, Ridge, Lasso, Logistic Regression, Naive Bayes, Decision Tree, Random Forest, Gradient Boosting, SVM, dan Neural Networks (MLP).
2. Menganalisis pengaruh regulasi L1/L2, kedalaman pohon, ukuran ensemble, dan fungsi aktivasi terhadap kompromi antara overfitting dan underfitting.
3. Menguasai teknik estimasi ketidakpastian model melalui `decision_function` dan `predict_proba`.
4. Menerapkan teknik prapemrosesan data pada Chapter 3 (StandardScaler, MinMaxScaler, RobustScaler, Normalizer) sekaligus menghindari kebocoran data (data leakage).

5. Memakai PCA, NMF, dan t-SNE untuk tiga keperluan berbeda: memampatkan data, menurunkan data berdimensi tinggi ke bidang yang bisa dilihat mata, dan menguraikan fitur menjadi komponen penyusunnya.
6. Mengevaluasi performa algoritma pengelompokan (k-Means, Agglomerative Hierarchical Clustering, DBSCAN) menggunakan metrik terstandarisasi Adjusted Rand Index dan Normalized Mutual Information.

1.3 Dataset dan Ruang Lingkup Eksperimen

Tujuh dataset standar dipakai sepanjang praktikum. Ketujuhanya sengaja dipilih agar mewakili spektrum persoalan yang berbeda, mulai dari data sintetis sederhana untuk menjelaskan konsep hingga data biomedis dan citra riil. Rinciannya disajikan pada Tabel 1.

Tabel 1. Daftar dataset yang digunakan dalam eksperimen.

Nama Dataset	Tipe Data	Dimensi / Sampel	Deskripsi & Peruntukan
Forge Dataset	Sintetis (2 Kelas)	(26, 2) (26 sampel, 2 fitur)	Visualisasi batas keputusan klasifikasi 2D
Wave Dataset	Sintetis (Kontinu)	40 sampel, 1 fitur	Visualisasi kurva regresi dan varians kompleksitas model
Breast Cancer Wisconsin	Biomedis Riil	(569, 30) (569 sampel, 30 fitur)	Klasifikasi biner tumor ganas (212) vs jinak (357)
Two Moons	Sintetis Non-Konveks	200 sampel, 2 fitur	Uji kemampuan clustering non-linear & manifold
Blobs Dataset	Sintetis Terpisah	3 klaster, 2 fitur	Uji dasar k-Means dan Agglomerative Clustering
Digits Dataset	Citra Riil 8x8	(1797, 64) (1.797 sampel, 64 fitur)	Uji PCA, t-SNE, dan clustering pada data berdimensi tinggi
Synthetic Signals	Gelombang Sinyal	2000 sampel, 3 komponen	Uji pemisahan sumber sinyal independen dengan NMF

BAB II SUPERVISED LEARNING

2.1 Konsep Dasar dan Eksplorasi Dataset

Supervised learning bekerja dengan mencari pola hubungan matematis antara vektor fitur masukan X dan variabel target y . Tantangan utamanya terletak pada generalisasi, yaitu kemampuan model memprediksi data baru yang belum pernah dilihat secara akurat. Model harus lolos dari dua jebakan sekaligus: overfitting, ketika model justru menghafal noise pada data pelatihan, dan underfitting, ketika model terlalu sederhana sehingga pola datanya tidak tertangkap.

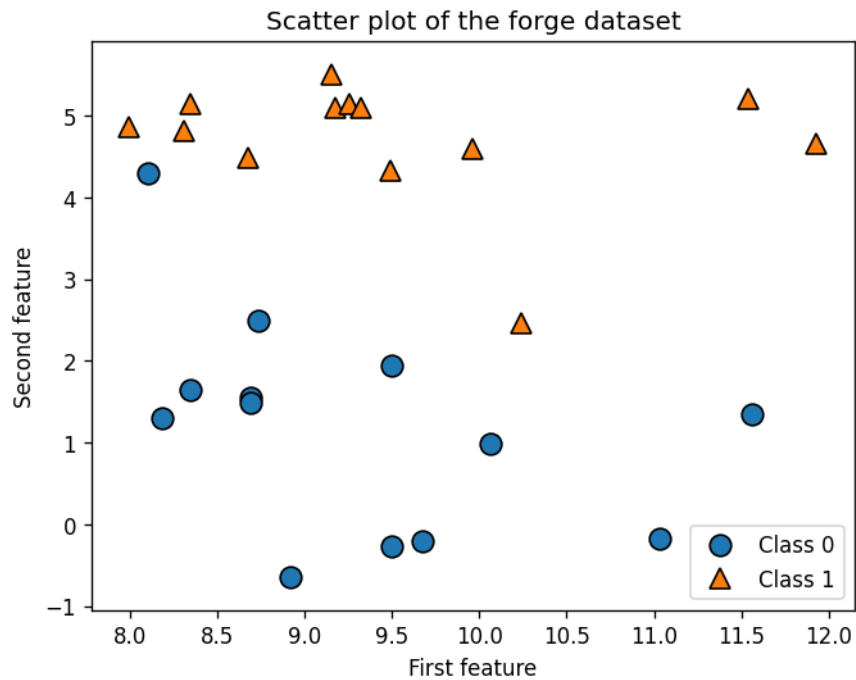
Kode 2.1: Eksplorasi Dataset Sintetis dan Dataset Medis Riil

```
# Eksplorasi Dataset Forge, Wave, dan Breast Cancer
import mglearn
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

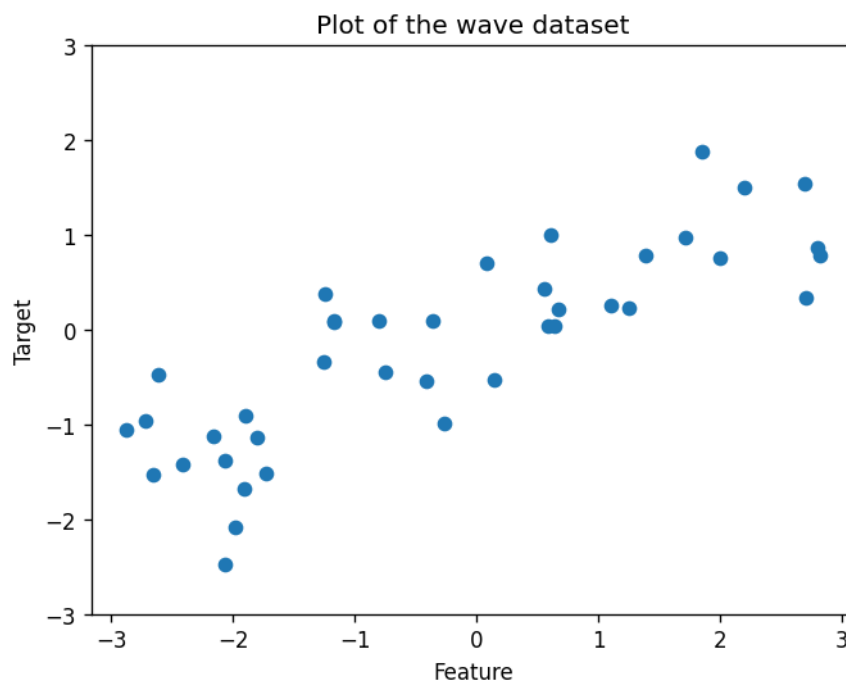
# 1. Dataset Forge (Klasifikasi Biner 2D)
X_forge, y_forge = mglearn.datasets.make_forge()
print("Forge shape:", X_forge.shape)

# 2. Dataset Wave (Regresi 1D)
X_wave, y_wave = mglearn.datasets.make_wave(n_samples=40)

# 3. Dataset Biomedis Breast Cancer Wisconsin
cancer = load_breast_cancer()
print("Cancer data shape:", cancer.data.shape)
print("Target classes:", cancer.target_names)
```



Gambar 2.1: Visualisasi Forge Dataset untuk Klasifikasi Biner Dua Fitur.



Gambar 2.2: Visualisasi Wave Dataset untuk Regresi Fitur Tunggal Kontinu.

Dataset klinis Breast Cancer Wisconsin memuat 569 sampel dengan 30 fitur numerik yang terbagi ke dalam dua kelas: 212 kasus malignant (ganas) dan 357 kasus benign (jinak). Rasio kelas yang timpang ini dijaga melalui stratified train-test split (75% latih, 25% uji) agar evaluasinya tetap adil dan objektif.

2.2 K-Nearest Neighbors (KNN)

KNN termasuk metode non-parametrik berbasis instan (lazy learning). Model eksplisit tidak pernah dibangun; sampel baru langsung diklasifikasikan menurut jarak terdekatnya, umumnya Euclidean distance, terhadap titik-titik pelatihan.

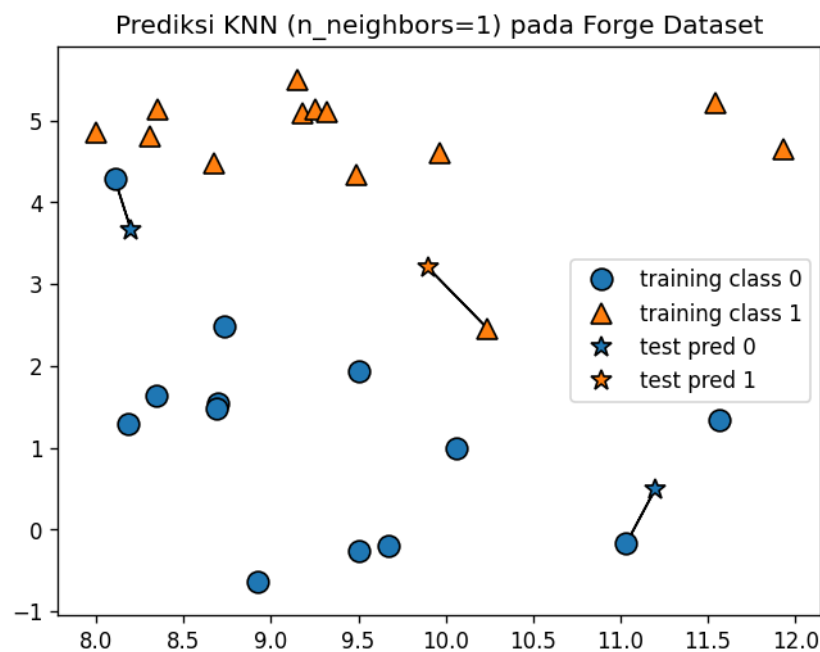
Kode 2.2: Pelatihan dan Evaluasi K-Nearest Neighbors Classifier

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split

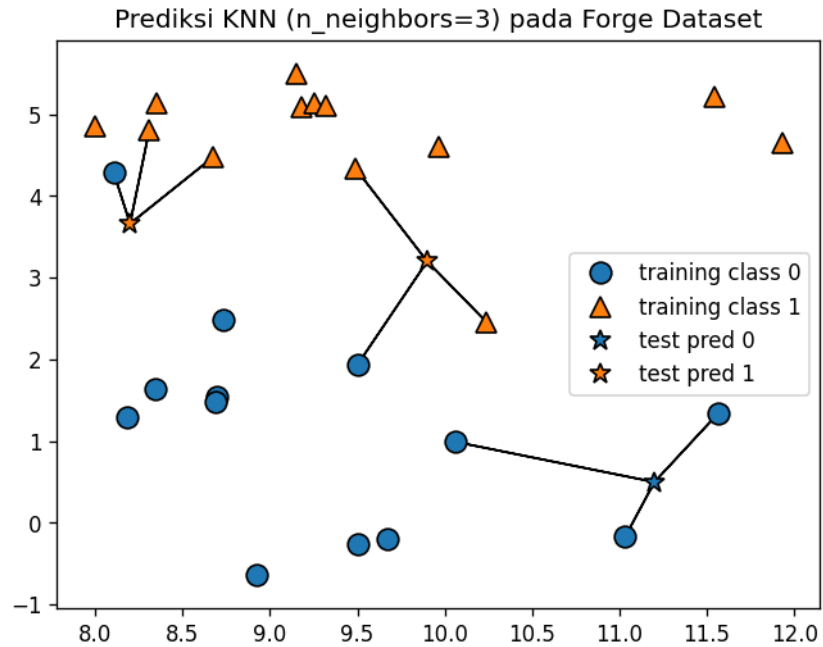
# Split dataset Forge
X_train, X_test, y_train, y_test = train_test_split(X_forge, y_forge, random_state=0)

# Pelatihan model KNN dengan k=3
clf = KNeighborsClassifier(n_neighbors=3)
clf.fit(X_train, y_train)

print(f"Prediksi Test Set: {clf.predict(X_test)}")
print(f"Akurasi Test Set (k=3): {clf.score(X_test, y_test):.2f}")
```

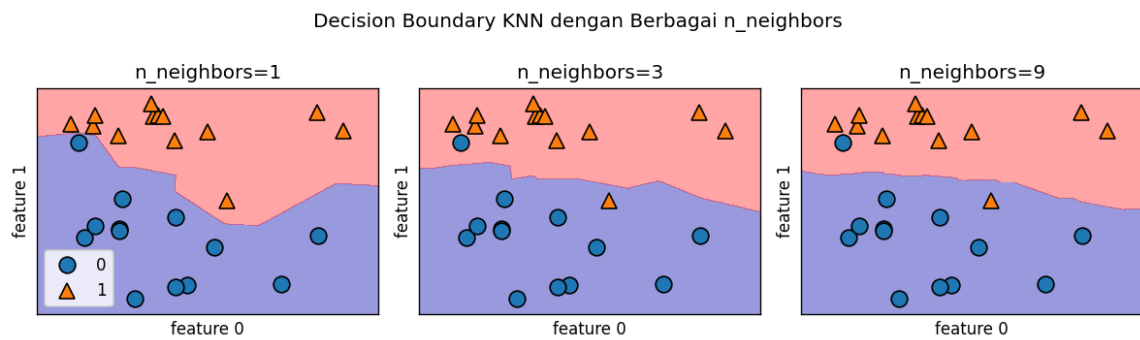


Gambar 2.3: Mekanisme Prediksi KNN dengan $k=1$ pada Forge Dataset.



Gambar 2.4: Mekanisme Prediksi KNN dengan $k=3$ Menggunakan Voting Mayoritas.

Pada Forge dataset, evaluasi test set menghasilkan akurasi 85.71% untuk $k=3$. Gambar 2.5 memperlihatkan bagaimana batas keputusan (decision boundary) berubah seiring bertambahnya jumlah tetangga k .



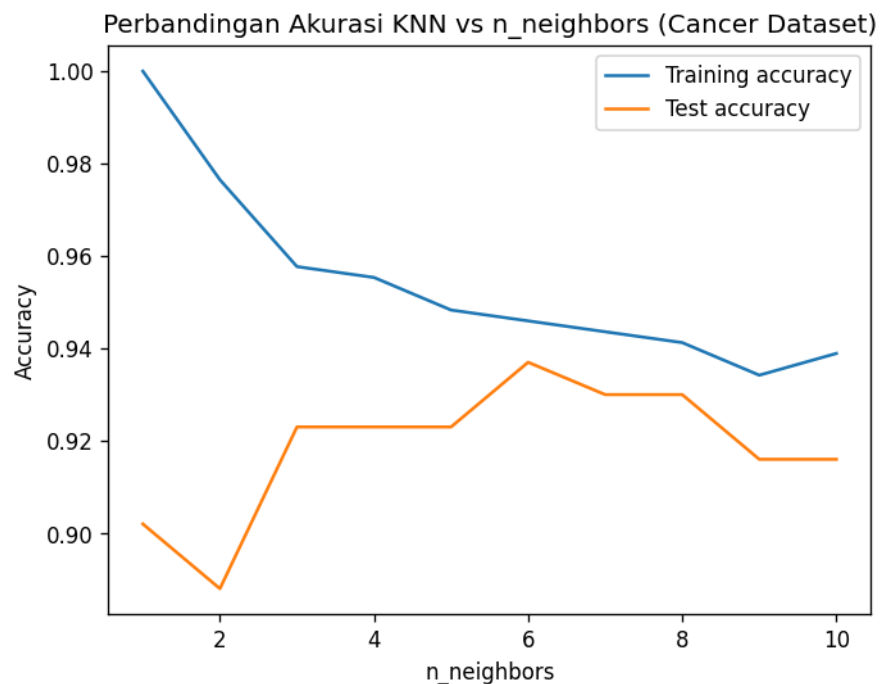
Gambar 2.5: Perubahan Decision Boundary KNN dari Fleksibel ($k=1$) Menjadi Halus ($k=9$).

Kode 2.3: Evaluasi Kurva Akurasi Train vs Test pada Variasi Nilai k

```
# Investigasi Trade-off Kompleksitas Model KNN pada Breast Cancer Dataset
X_train_c, X_test_c, y_train_c, y_test_c = train_test_split(
    cancer.data, cancer.target, stratify=cancer.target, random_state=66)

training_accuracy, test_accuracy = [], []
neighbors_settings = range(1, 11)

for n_neighbors in neighbors_settings:
    clf_n = KNeighborsClassifier(n_neighbors=n_neighbors)
    clf_n.fit(X_train_c, y_train_c)
    training_accuracy.append(clf_n.score(X_train_c, y_train_c))
    test_accuracy.append(clf_n.score(X_test_c, y_test_c))
```



Gambar 2.6: Kurva Akurasi Train vs Test pada Breast Cancer terhadap Variasi Nilai k.

Analisis kompromi pada KNN. Titik optimal berada di $k = 6$ dengan akurasi pengujian tertinggi 93.71%. Pada $k=1$ muncul overfitting ekstrem: akurasi latih menyentuh 100% sementara akurasi ujinya rendah. Sebaliknya, saat k mendekati 10 batas keputusan menjadi terlalu kaku sehingga akurasi latih dan uji sama-sama menurun.

Regresi dengan K-Nearest Neighbors

Untuk kasus regresi pada Wave dataset, nilai target diperoleh dari rata-rata target tetangga terdekat. KNN Regressor dengan $k=3$ menghasilkan koefisien determinasi R^2 pengujian sebesar 0.8344. Perbandingan performa R^2 terhadap variasi k disajikan pada Tabel 2.

Kode 2.4: Implementasi KNN Regression pada Wave Dataset

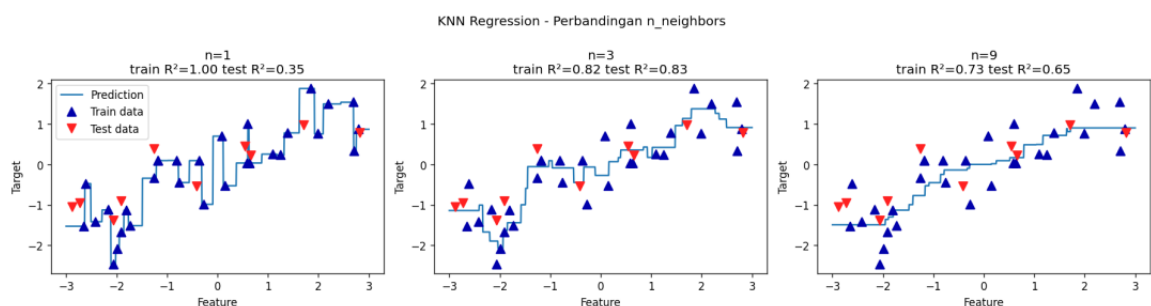
```
from sklearn.neighbors import KNeighborsRegressor

X_train_r, X_test_r, y_train_r, y_test_r = train_test_split(X_wave, y_wave,
random_state=0)

# KNN Regressor dengan variasi jumlah tetangga
for k in [1, 3, 9]:
    reg = KNeighborsRegressor(n_neighbors=k)
    reg.fit(X_train_r, y_train_r)
    print(f"k={k} -> Train R2: {reg.score(X_train_r, y_train_r):.3f} | "
          f"Test R2: {reg.score(X_test_r, y_test_r):.3f}")
```

Tabel 2. Skor R^2 KNN Regressor pada Wave dataset untuk variasi nilai k .

Jumlah Tetangga (k)	R^2 Skor Pelatihan (Train)	R^2 Skor Pengujian (Test)
$k = 1$	1.000	0.352
$k = 3$	0.819	0.834
$k = 9$	0.728	0.654



Gambar 2.7: Prediksi KNN Regressor pada Wave Dataset untuk $k=1$, $k=3$, dan $k=9$.

2.3 Model Linier untuk Regresi: Linear, Ridge, dan Lasso

Ordinary Least Squares (OLS) mencari garis yang membuat jumlah kuadrat selisih antara nilai sebenarnya dan nilai prediksi sekecil mungkin. Persoalan muncul

ketika jumlah fiturnya membengkak. Pada versi dataset yang sudah diperluas menjadi 104 fitur interaksi polynomial, OLS punya cukup kebebasan untuk menempel nyaris sempurna pada data latih, lalu kehilangan arah pada data baru: skor pelatihannya 95.21%, sedangkan skor pengujiannya jatuh ke 60.75%.

Kode 2.5: Komparasi Regresi Linier OLS, Ridge (L2), dan Lasso (L1)

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso

# Menggunakan dataset berdimensi tinggi (Extended Boston - 104 fitur)
X_lr, y_lr = mglearn.datasets.load_extended_boston()
X_tr, X_te, y_tr, y_te = train_test_split(X_lr, y_lr, random_state=0)

# 1. Linear Regression OLS
lr = LinearRegression().fit(X_tr, y_tr)
print(f"OLS -> Train R2: {lr.score(X_tr, y_tr):.2f} | Test R2: {lr.score(X_te, y_te):.2f}")

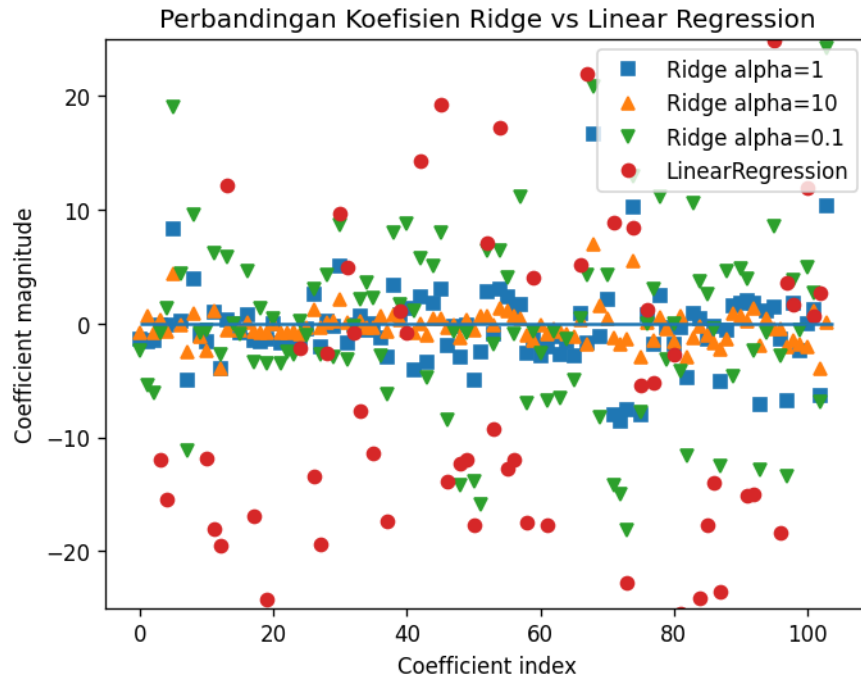
# 2. Ridge Regression (L2 Regularization)
ridge = Ridge(alpha=1.0).fit(X_tr, y_tr)
ridge01 = Ridge(alpha=0.1).fit(X_tr, y_tr)
ridge10 = Ridge(alpha=10.0).fit(X_tr, y_tr)

# 3. Lasso Regression (L1 Regularization & Feature Selection)
lasso = Lasso(alpha=1.0).fit(X_tr, y_tr)
lasso001 = Lasso(alpha=0.01, max_iter=100000).fit(X_tr, y_tr)
```

Ridge Regression (Regularisasi L2)

Ridge meredam overfitting dengan menambahkan penalti L2 pada magnitudo koefisien ($\alpha * \|w\|^2$). Penalti ini menarik koefisien mendekati nol tanpa pernah benar-benar menghapusnya, sebuah mekanisme yang dikenal sebagai weight decay. Hasil eksperimennya:

- Ridge (alpha=0.1): Skor Train = 92.82%, Skor Test = 77.22% (generalisasi optimal)
- Ridge (alpha=1.0): Skor Train = 88.58%, Skor Test = 75.28% (keseimbangan standar)
- Ridge (alpha=10.0): Skor Train = 78.83%, Skor Test = 63.59% (terlalu terbatas, underfitting)



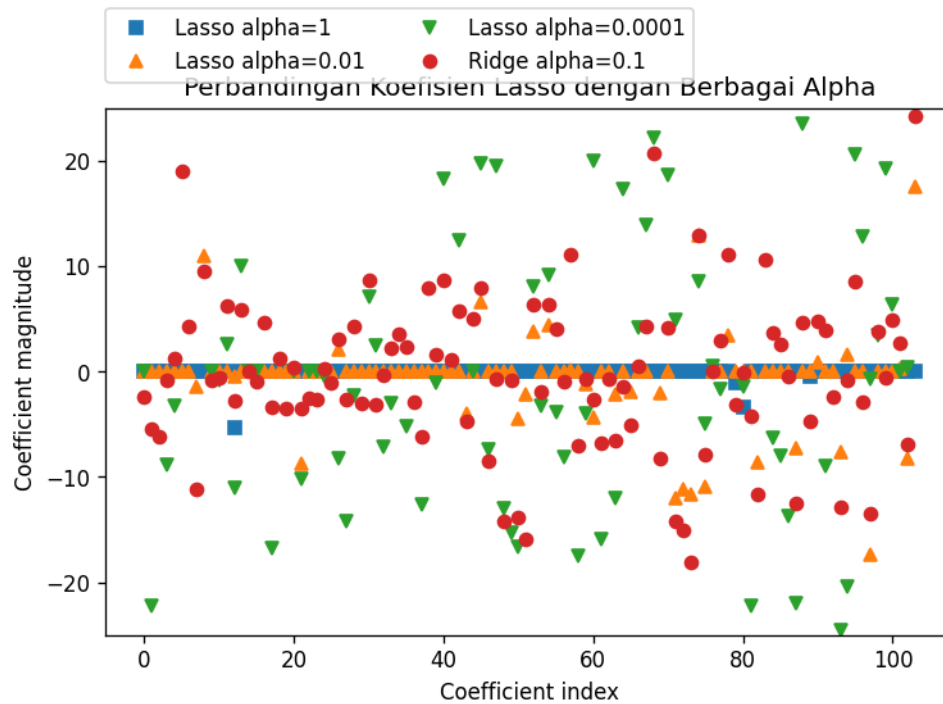
Gambar 2.8: Distribusi Besaran Koefisien Ridge Regression terhadap Variasi Alpha.

Lasso Regression (Regularisasi L1 dan Seleksi Fitur)

Lasso memakai penalti L1 ($\alpha * |w|$). Karakteristik matematisnya unik: koefisien dapat ditekan hingga bernilai tepat nol. Sifat inilah yang membuat Lasso berfungsi sekaligus sebagai metode seleksi fitur otomatis (automatic feature selection). Perbandingan hasilnya ada pada Tabel 3.

Tabel 3. Pengaruh parameter alpha pada Lasso Regression.

Parameter Alpha	R ² Train Score	R ² Test Score	Jumlah Fitur Digunakan (Non-Zero)
alpha = 1.0	29.32%	20.94%	4 fitur (underfitting parah)
alpha = 0.01	89.62%	76.57%	33 fitur (trade-off optimal)
alpha = 0.0001	95.07%	64.37%	96 fitur (mulai overfitting)



Gambar 2.9: Penyusutan Koefisien Menjadi Nol pada Model Lasso Regression.

2.4 Klasifikasi Linier dan Logistic Regression

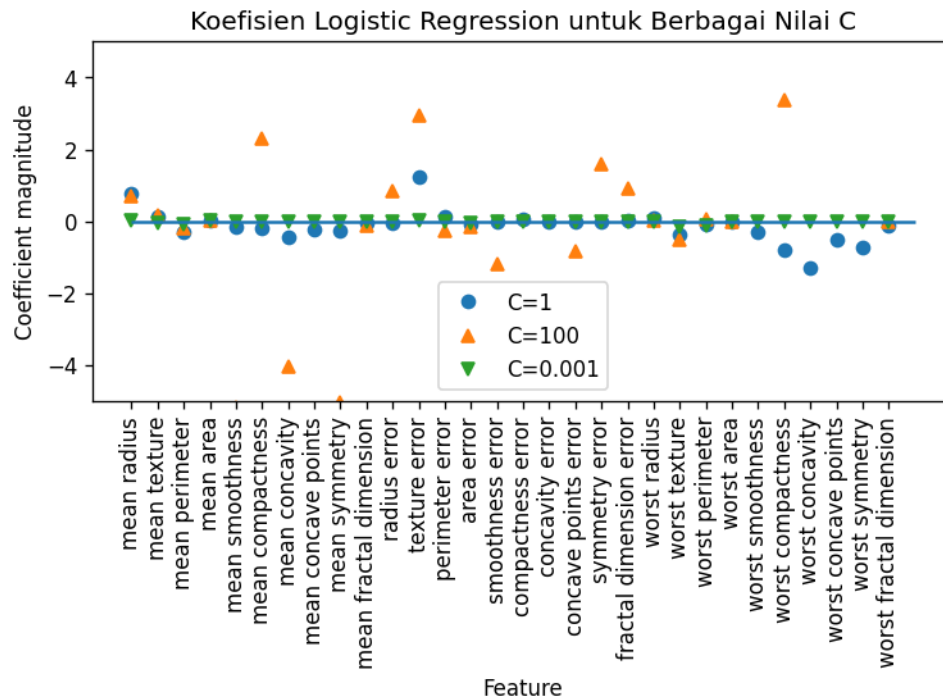
Logistic Regression memodelkan probabilitas keanggotaan kelas lewat fungsi sigmoid logistik. Parameter C berperan sebagai kebalikan dari kekuatan regulasi ($C = 1/\alpha$). Nilai C yang besar berarti penalti kecil, sehingga model berusaha mencocokkan setiap titik sampel; nilai C yang kecil justru mendorong koefisien mendekati nol.

Kode 2.6: Logistic Regression dengan Berbagai Nilai Parameter C

```
from sklearn.linear_model import LogisticRegression

# Klasifikasi Biner pada Dataset Breast Cancer
logreg = LogisticRegression(max_iter=5000).fit(X_train_c, y_train_c)
logreg100 = LogisticRegression(C=100, max_iter=5000).fit(X_train_c, y_train_c)
logreg001 = LogisticRegression(C=0.01, max_iter=5000).fit(X_train_c, y_train_c)

print(f"C=1    -> Train: {logreg.score(X_train_c, y_train_c):.3f} | "
      f"Test: {logreg.score(X_test_c, y_test_c):.3f}")
print(f"C=100  -> Train: {logreg100.score(X_train_c, y_train_c):.3f} | "
      f"Test: {logreg100.score(X_test_c, y_test_c):.3f}")
print(f"C=0.01 -> Train: {logreg001.score(X_train_c, y_train_c):.3f} | "
      f"Test: {logreg001.score(X_test_c, y_test_c):.3f}")
```



Gambar 2.10: Profil Koefisien Logistic Regression pada Breast Cancer untuk $C=0.01$, $C=1$, dan $C=100$.

Pengujian pada Breast Cancer memperlihatkan akurasi naik dari 95.10% ($C=0.01$) menjadi 95.80% ($C=1$), lalu mencapai 96.50% pada $C=100$.

2.5 Naive Bayes Classifiers

Naive Bayes bekerja di atas Teorema Bayes dengan asumsi 'naif' bahwa setiap fitur saling bebas (independent) secara kondisional terhadap kelas target. Pada GaussianNB, yang mengasumsikan distribusi normal untuk fitur kontinu, pelatihan berlangsung nyaris instan dengan akurasi latih 95.07% dan akurasi uji 93.71%. Model ini tangguh sekaligus hemat komputasi, terutama untuk data bervolume besar.

Kode 2.7: Implementasi Gaussian Naive Bayes Classifier

```
from sklearn.naive_bayes import GaussianNB

# Pelatihan Gaussian Naive Bayes pada Breast Cancer
gnb = GaussianNB().fit(X_train_c, y_train_c)
print(f"GNB Training Accuracy: {gnb.score(X_train_c, y_train_c):.3f}")
print(f"GNB Test Accuracy: {gnb.score(X_test_c, y_test_c):.3f}")
```

2.6 Decision Tree dan Analisis Feature Importance

Decision Tree menyusun struktur pohon biner hierarkis melalui partisi fitur yang meminimalkan impurity (Gini atau Entropy). Pohon tanpa batasan kedalaman (unpruned) rentan overfitting ekstrem karena terus membelah node sampai seluruh daunnya murni (pure leaves). Pohon penuh mencatat akurasi latih 100.0%, namun akurasi ujinya tertahan di 93.71%.

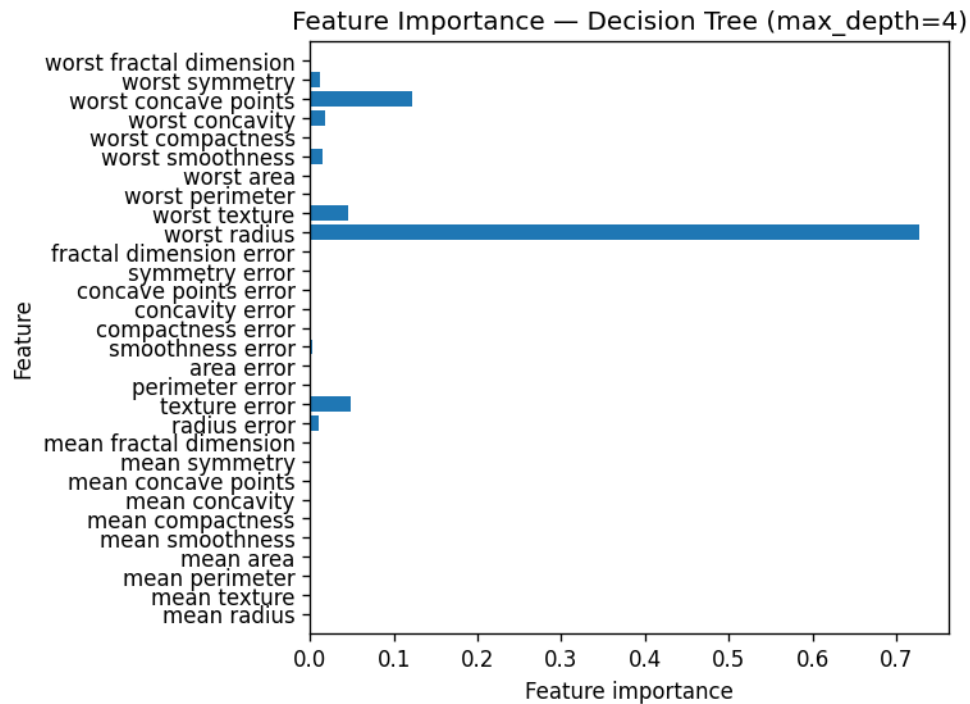
Kode 2.8: Decision Tree Classifier dan Pengaruh Pre-Pruning

```
from sklearn.tree import DecisionTreeClassifier, plot_tree

# 1. Unpruned Tree (Overfitting)
tree_full = DecisionTreeClassifier(random_state=0).fit(X_train_c, y_train_c)
print(f"Full Tree -> Train: {tree_full.score(X_train_c, y_train_c):.3f} | "
      f"Test: {tree_full.score(X_test_c, y_test_c):.3f}")

# 2. Pre-pruned Tree (max_depth=4)
tree_pruned = DecisionTreeClassifier(max_depth=4, random_state=0).fit(X_train_c,
y_train_c)
print(f"Depth=4 -> Train: {tree_pruned.score(X_train_c, y_train_c):.3f} | "
      f"Test: {tree_pruned.score(X_test_c, y_test_c):.3f}")

# Feature importances
print("Fitur paling penting:",
      cancer.feature_names[np.argmax(tree_pruned.feature_importances_)])
```



Gambar 2.11: Nilai Feature Importance Pohon Keputusan (Fitur 'worst radius' Dominan).

2.7 Ensemble Pohon Keputusan: Random Forest dan Gradient Boosting

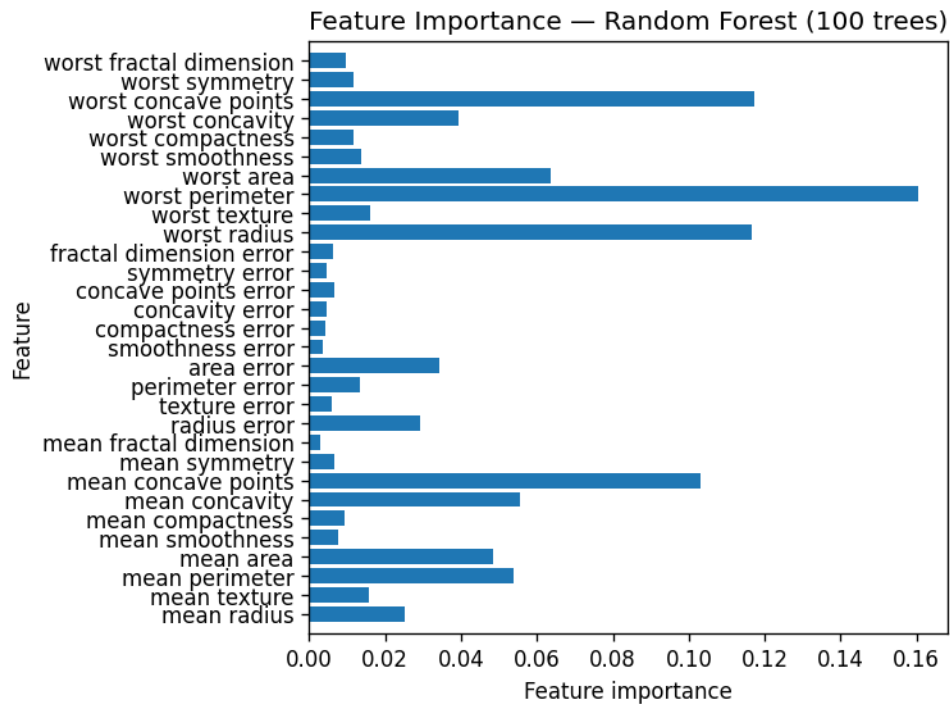
Metode ensemble menggabungkan prediksi dari banyak pohon keputusan untuk menghasilkan model yang lebih kokoh dan stabil dibanding pohon tunggal.

Kode 2.9: Implementasi Ensemble Random Forest dan Gradient Boosting

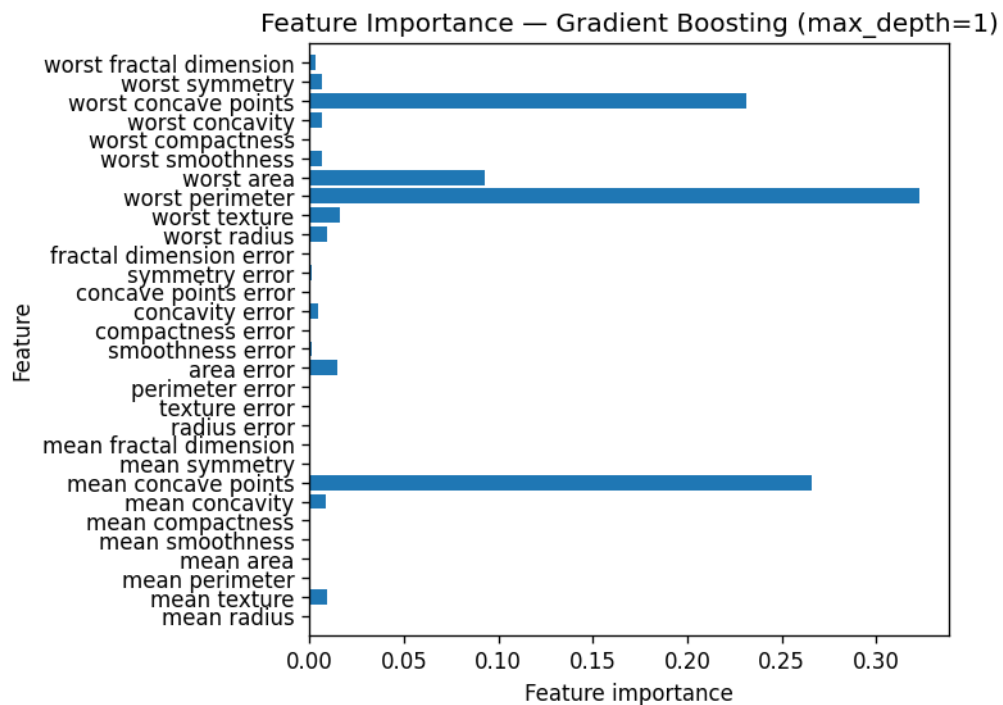
```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier

# 1. Random Forest (100 Decision Trees via Bagging + Feature Subsampling)
rf = RandomForestClassifier(n_estimators=100, random_state=0).fit(X_train_c,
y_train_c)
print(f"Random Forest -> Train: {rf.score(X_train_c, y_train_c):.3f} | "
      f"Test: {rf.score(X_test_c, y_test_c):.3f}")

# 2. Gradient Boosted Regression Trees (Sequential Error Correction)
gbrt = GradientBoostingClassifier(random_state=0, max_depth=1).fit(X_train_c,
y_train_c)
print(f"GBRT (depth=1) -> Train: {gbrt.score(X_train_c, y_train_c):.3f} | "
      f"Test: {gbrt.score(X_test_c, y_test_c):.3f}")
```



Gambar 2.12: Feature Importance Random Forest (Distribusi Kepentingan Fitur Lebih Merata).



Gambar 2.13: Feature Importance GBRT yang Menitikberatkan Fitur-Fitur Kunci Secara Efisien.

2.8 Kernelized Support Vector Machines (SVM)

Data yang tidak terpisahkan oleh sebuah garis lurus di ruang aslinya sering kali terpisah rapi begitu diangkat ke ruang berdimensi lebih tinggi. Kernel SVM melakukan pengangkatan itu tanpa pernah menghitung koordinat barunya secara eksplisit, sebuah

jalan pintas yang dikenal sebagai kernel trik, dengan Radial Basis Function (RBF) sebagai kernel yang paling lazim dipakai. Di ruang tersebut SVM mencari bidang pemisah yang jarak marginnya ke kedua kelas paling lebar. Titik lemahnya jelas: karena seluruh perhitungan bertumpu pada jarak, perbedaan skala antarfitur langsung merusak hasilnya.

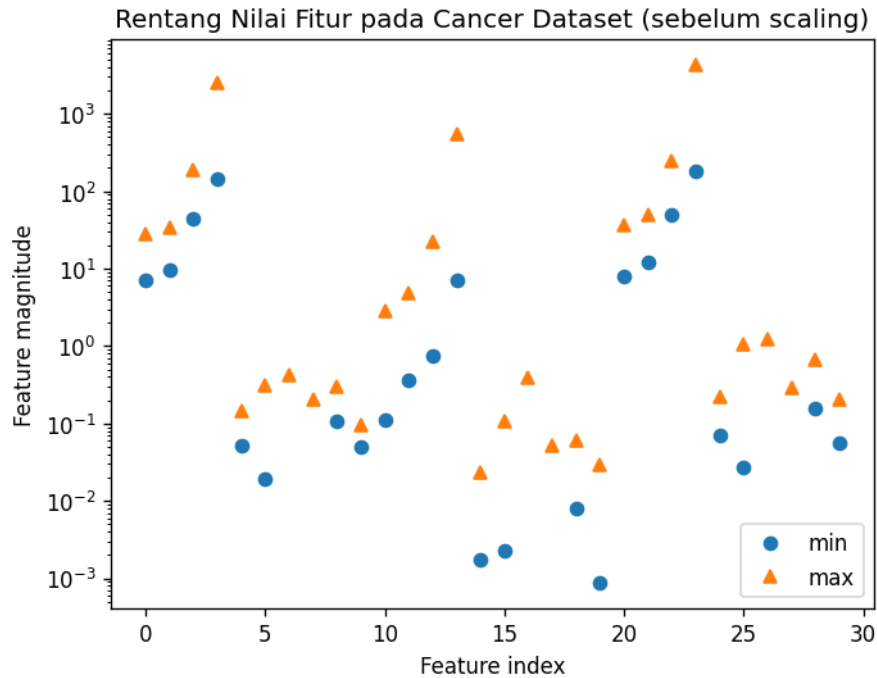
Kode 2.10: Penanganan Disparitas Skala Fitur pada Support Vector Machine

```
from sklearn.svm import SVC
from sklearn.preprocessing import MinMaxScaler

# 1. SVM pada data mentah (tanpa scaling)
svc_raw = SVC().fit(X_train_c, y_train_c)
print(f"SVM Raw    -> Train: {svc_raw.score(X_train_c, y_train_c):.3f} | "
      f"Test: {svc_raw.score(X_test_c, y_test_c):.3f}")

# 2. SVM dengan Normalisasi MinMaxScaler [0, 1]
scaler = MinMaxScaler().fit(X_train_c)
X_train_scaled = scaler.transform(X_train_c)
X_test_scaled = scaler.transform(X_test_c)

svc_scaled = SVC(C=1000).fit(X_train_scaled, y_train_c)
print(f"SVM Scaled -> Train: {svc_scaled.score(X_train_scaled, y_train_c):.3f} | "
      f"Test: {svc_scaled.score(X_test_scaled, y_test_c):.3f}")
```



Gambar 2.14: Disparitas Skala Fitur Ekstrem pada Dataset Breast Cancer Tanpa Penskalaan.

Tanpa penskalaan, SVM RBF hanya sampai di akurasi uji 93.71%. Setelah fitur diskalakan ke rentang $[0, 1]$ dengan MinMaxScaler, akurasinya melonjak tajam ke 97.20%. Bukti ini menegaskan prapemrosesan sebagai prasyarat bagi algoritma yang bertumpu pada margin dan jarak geometris.

2.9 Neural Networks (Multilayer Perceptron atau MLP)

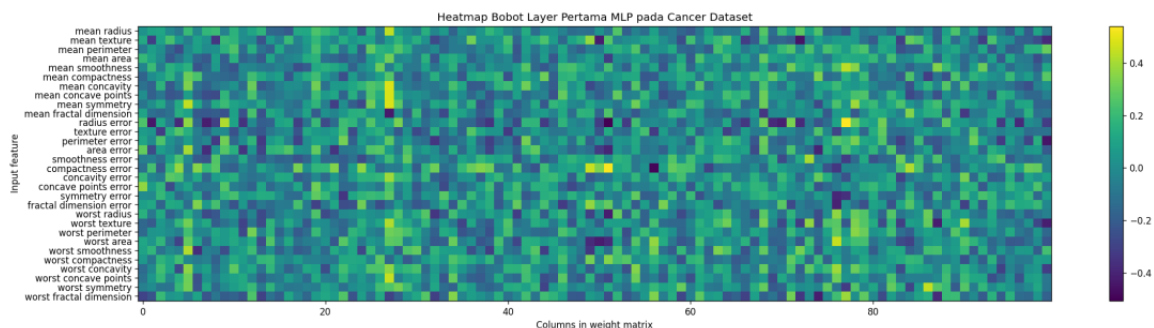
MLP menyalurkan data melewati beberapa lapis neuron tersembunyi. Di setiap lapis, kombinasi linear bobot dilewatkan ke fungsi aktivasi non-linear ReLU, dan penumpukan lapis itulah yang memberi jaringan kemampuan memodelkan hubungan yang jauh lebih rumit daripada satu bidang pemisah. Pada eksperimen Breast Cancer dengan fitur yang sudah diskalakan, MLP menyentuh akurasi uji 97.20%. Batas iterasi yang dinaikkan dari 100 ke 1000 memberi proses optimasi cukup waktu untuk mencapai konvergensi bobot secara penuh.

Kode 2.11: Arsitektur Multi-Layer Perceptron (MLP) dengan Regularisasi L2

```
from sklearn.neural_network import MLPClassifier
from sklearn.preprocessing import StandardScaler

# Normalisasi Z-Score untuk Jaringan Syaraf Tiruan
scaler_std = StandardScaler().fit(X_train_c)
X_train_std = scaler_std.transform(X_train_c)
X_test_std = scaler_std.transform(X_test_c)

# MLP Classifier dengan 2 Hidden Layer (masing-masing 100 neuron)
mlp = MLPClassifier(hidden_layer_sizes=[100, 100], max_iter=1000, alpha=1.0,
                    random_state=42)
mlp.fit(X_train_std, y_train_c)
print(f"MLP (Iter=1000) -> Train: {mlp.score(X_train_std, y_train_c):.3f} | "
      f"Test: {mlp.score(X_test_std, y_test_c):.3f}")
```



Gambar 2.15: Heatmap Visualisasi Bobot Lapisan Pertama Jaringan Syaraf Tiruan MLP.

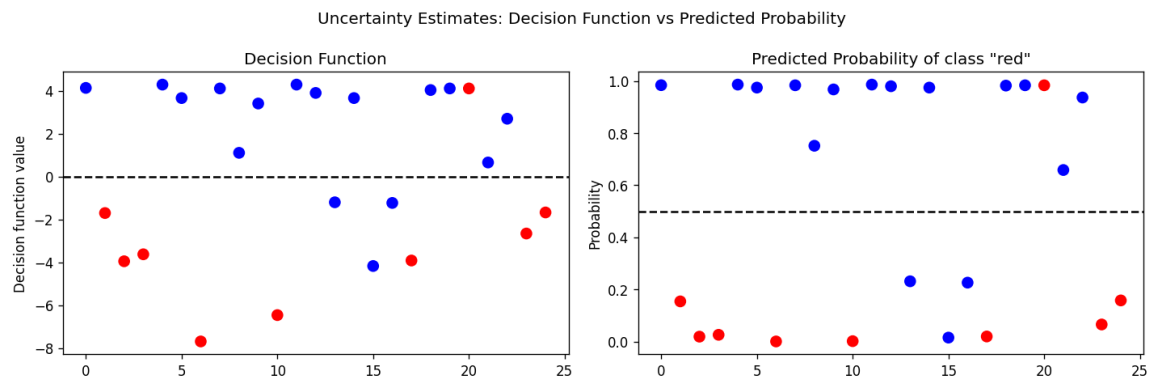
2.10 Estimasi Ketidakpastian (Uncertainty Estimates)

Scikit-Learn menyediakan dua antarmuka untuk mengukur tingkat keyakinan prediksi:

1. `decision_function` mengembalikan satu bilangan bertanda yang menyatakan sejauh mana dan ke sisi mana sebuah sampel berada relatif terhadap bidang pemisah. Semakin besar nilai mutlak nya, semakin yakin model tersebut. Pada eksperimen ini nilainya terbentang dari -7.69 sampai +4.29.
2. `predict_proba` mengubah keyakinan itu menjadi probabilitas posterior yang sudah dinormalisasi ke rentang $[0, 1]$, dan totalnya untuk seluruh kelas selalu berjumlah 1. Bentuk ini lebih mudah dibaca ketika ambang keputusan perlu digeser.

Kode 2.12: Ekstraksi Decision Function dan Predict Proba

```
# Pengukuran Ketidakpastian Prediksi Model
print("Decision Function (6 sampel pertama):\n",
      gbdt.decision_function(X_test_c)[:6])
print("\nPredicted Probabilities (6 sampel pertama):\n",
      gbdt.predict_proba(X_test_c)[:6])
```



Gambar 2.16: Perbandingan Decision Function vs Predicted Probability pada Gradient Boosting.

2.11 Rangkuman Komparatif Performa Supervised Learning

Tabel 4 merangkum performa empiris seluruh model yang diuji pada dataset Breast Cancer Wisconsin.

Tabel 4. Rangkuman performa model supervised learning pada Breast Cancer Wisconsin.

Algoritma Model	Skor Pelatihan (Train)	Skor Pengujian (Test)	Karakteristik & Trade-off Utama
K-Nearest Neighbors (k=6)	94.60%	93.71%	Sederhana, memori besar, sensitif fitur irelevan
Logistic Regression (C=100)	98.12%	96.50%	Cepat, terinterpretasi tinggi, linier
Gaussian Naive Bayes	95.07%	93.71%	Sangat cepat, asumsi fitur independen
Decision Tree (max_depth=4)	98.83%	95.10%	Mudah divisualisasi, rawan overfitting tanpa pruning

Algoritma Model	Skor Pelatihan (Train)	Skor Pengujian (Test)	Karakteristik & Trade-off Utama
Random Forest (100 trees)	100.00%	97.20%	Sangat stabil, kuat terhadap noise, performa papan atas
Gradient Boosted Trees (depth=1)	99.06%	97.20%	Akurasi tinggi, kompak, sensitif hyperparameter
Kernel SVM (RBF, MinMax)	98.40%	97.20%	Sangat kuat untuk data non-linear berdimensi sedang
Multi-Layer Perceptron (MLP)	100.00%	97.20%	Menangkap representasi kompleks, butuh waktu tuning
Linear Regression (OLS)	95.21%	60.75%	Overfitting parah pada data banyak fitur interaksi

BAB III UNSUPERVISED LEARNING DAN PRAPEMROSESAN DATA

3.1 Penskalaan Fitur dan Bahaya Kebocoran Data

Unsupervised learning menggali struktur intrinsik data tanpa panduan label target y. Tahap pertama yang menentukan dalam pipeline machine learning adalah transformasi dan penskalaan fitur. Scikit-Learn menyediakan empat transformer utama untuk keperluan ini:

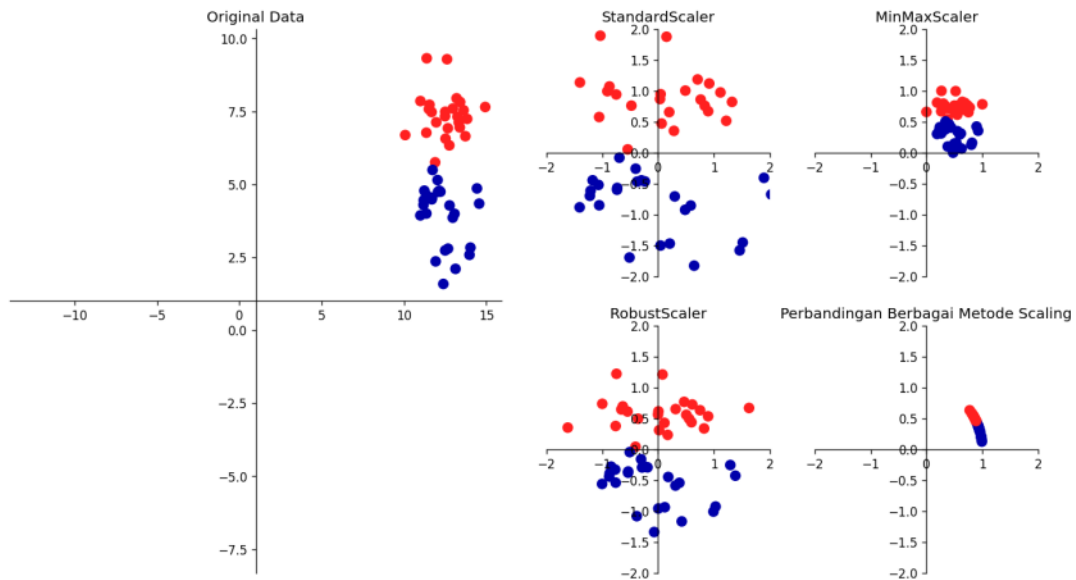
1. **StandardScaler**: memusatkan distribusi tiap fitur pada rata-rata nol lalu membaginya dengan simpangan bakunya, sehingga seluruh fitur berbagi satuan statistik yang sama (z-score).
2. **RobustScaler**: bersandar pada median dan rentang interkuartil, bukan rata-rata dan simpangan baku. Nilai pencilan yang jauh karena itu tidak ikut menarik pusat maupun rentang skalanya.
3. **MinMaxScaler**: memetakan nilai terkecil dan terbesar tiap fitur ke batas yang ditentukan, yang secara baku jatuh pada 0 dan 1.
4. **Normalizer**: bekerja per baris, bukan per kolom. Tiap vektor sampel diperpendek atau diperpanjang sampai norma Euclidean-nya bernilai satu, sehingga yang tersisa hanyalah informasi arah.

Kode 3.1: Empat Metode Penskalaan Fitur Standar Scikit-Learn

```
from sklearn.preprocessing import (StandardScaler, MinMaxScaler,
                                   RobustScaler, Normalizer)

# Inisialisasi 4 transformer penskalaan data
scalers = {
    'StandardScaler': StandardScaler(),
    'MinMaxScaler': MinMaxScaler(),
    'RobustScaler': RobustScaler(),
    'Normalizer': Normalizer()
}

# Contoh transformasi pada dataset
for name, s in scalers.items():
    X_trans = s.fit_transform(cancer.data)
    print(f"{name:15s} -> Min: {X_trans.min():.2f}, "
          f"Max: {X_trans.max():.2f}")
```



Gambar 3.1: Perbandingan Karakteristik Distribusi Geometris Empat Metode Penskalaan Data.

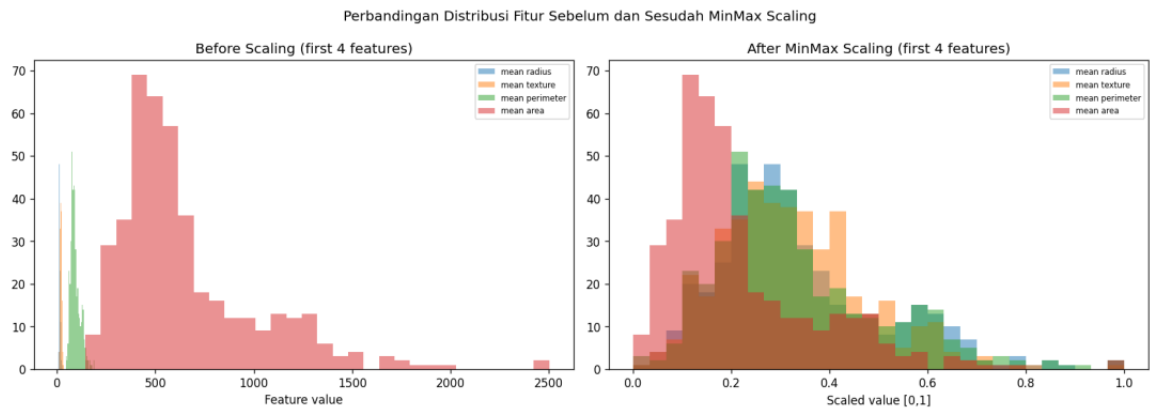
Bahaya Data Leakage pada Transformasi Test Set

Satu kekeliruan kecil pada tahap ini bisa membatalkan seluruh hasil evaluasi: memanggil `fit_transform` pada data uji. Parameter skala, baik itu nilai minimum dan maksimum maupun rata-rata dan simpangan baku, hanya boleh dipelajari dari porsi latih. Data uji cukup ditransformasikan memakai parameter yang sudah terlanjur dipelajari itu. Begitu data uji diskalakan memakai statistiknya sendiri, dua hal terjadi sekaligus: posisi relatif antara kedua himpunan tidak lagi sebanding, dan model secara tidak langsung sudah mengintip informasi yang seharusnya belum ia ketahui. Inilah yang disebut kebocoran data.

Kode 3.2: Pencegahan Kebocoran Data (Data Leakage) pada Preprocessing

```
# Pipeline Preprocessing yang Benar (Menghindari Data Leakage)
scaler = MinMaxScaler()
scaler.fit(X_train_c)    # FIT HANYA PADA TRAIN SET

X_train_scaled = scaler.transform(X_train_c)
X_test_scaled = scaler.transform(X_test_c)    # TRANSFORM TEST SET DENGAN PARAMETER
TRAIN
```



Gambar 3.2: Histogram Penskalaan Fitur Sebelum dan Sesudah MinMaxScaler.

Dampak prapemrosesan. Bukti empirisnya jelas. Akurasi model SVM pada data mentah tanpa penskalaan berada di 94.41%. Setelah MinMaxScaler diterapkan, akurasi naik menjadi 96.50%. Peningkatan sebesar itu diperoleh semata-mata dari normalisasi data yang benar, tanpa mengubah algoritma maupun hyperparameter.

3.2 Reduksi Dimensi: Principal Component Analysis (PCA)

PCA adalah teknik transformasi ortogonal linear yang merotasi sumbu koordinat data menuju arah yang memaksimalkan varians. Pada dataset Breast Cancer, PCA memampatkan 30 dimensi fitur menjadi 2 komponen utama saja, setara reduksi 93.3% dimensi.

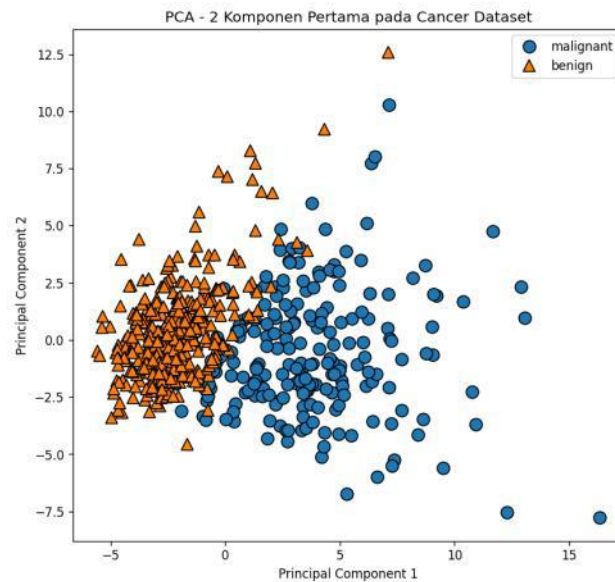
Kode 3.3: Dekomposisi dan Reduksi Dimensi dengan PCA

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# Standarisasi data sebelum PCA (Prasyarat Wajib)
X_cancer_scaled = StandardScaler().fit_transform(cancer.data)

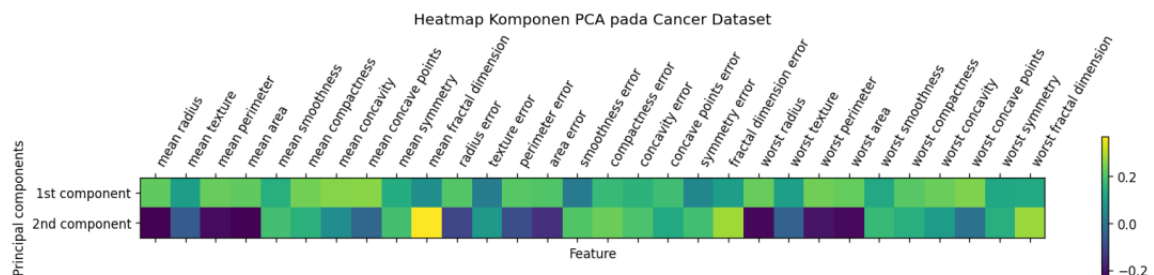
# Reduksi dimensi menjadi 2 Komponen Utama (PC1 & PC2)
pca = PCA(n_components=2)
pca.fit(X_cancer_scaled)
X_pca = pca.transform(X_cancer_scaled)

print("Explained Variance Ratio:", pca.explained_variance_ratio_)
print(f"Total Varians Terekstraksi:
{np.sum(pca.explained_variance_ratio_)*100:.2f}%")
```



Gambar 3.3: Proyeksi 2D PCA pada Breast Cancer (Separasi Kelas Tumor Malignant vs Benign Sangat Jelas).

Sumbu pertama hasil rotasi, PC1, sendirian menyerap 44.27% varians. PC2 menambah 18.97% lagi, sehingga keduanya bersama-sama menahan 63.24% keragaman data. Angka ini menyiratkan bahwa struktur pembeda antara tumor ganas dan jinak tidak tersebar merata di 30 fitur asli, melainkan menumpuk pada segelintir arah saja.



Gambar 3.4: Heatmap Beban Koefisien Fitur pada Komponen Utama PCA.

3.3 Non-Negative Matrix Factorization (NMF)

NMF memecah sebuah matriks data yang seluruh isinya tidak negatif menjadi hasil kali dua matriks berperingkat rendah, W dan H , dengan syarat tak satu pun elemennya boleh turun di bawah nol. Karena tidak ada suku negatif yang bisa saling meniadakan, komponen yang dihasilkan hanya dapat dijumlahkan. Itulah sebabnya NMF cocok untuk menyusun representasi berbasis bagian (part-based representation) dan untuk memisahkan sumber sinyal yang tercampur (blind source separation).

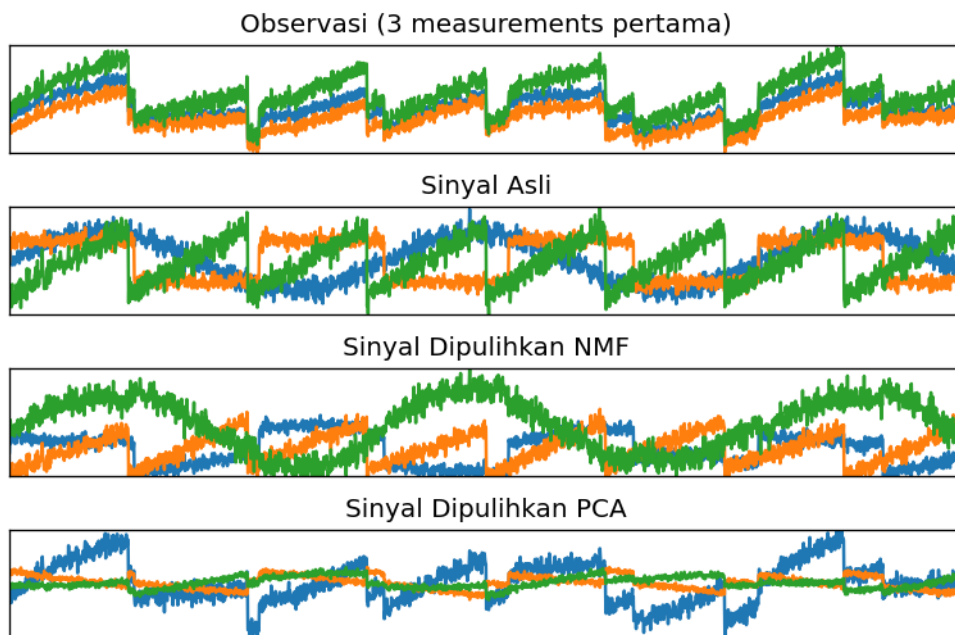
Kode 3.4: Ekstraksi Fitur Non-Negatif Menggunakan NMF

```
from sklearn.decomposition import NMF

# NMF untuk Pemisahan Sumber Sinyal Non-Negatif (Blind Source Separation)
nmf = NMF(n_components=3, random_state=42)
S_recovered = nmf.fit_transform(X_signals_mixed)

print("Dimensi Campuran Sinyal:", X_signals_mixed.shape)
print("Dimensi Sinyal Terpulihkan:", S_recovered.shape)
```

Perbandingan NMF vs PCA dalam Pemulihan Sinyal



Gambar 3.5: Keberhasilan NMF Memisahkan Campuran 3 Sinyal Audio Menjadi Komponen Aslinya.

3.4 Manifold Learning dengan t-SNE

t-Distributed Stochastic Neighbor Embedding (t-SNE) adalah algoritma manifold non-linear yang dirancang khusus untuk visualisasi. t-SNE mempertahankan kedekatan probabilitas antartetangga lokal dari ruang berdimensi tinggi ke dalam ruang dua dimensi. Eksperimen dilakukan pada Digits dataset yang berisi sampel citra angka berukuran 8x8 piksel.

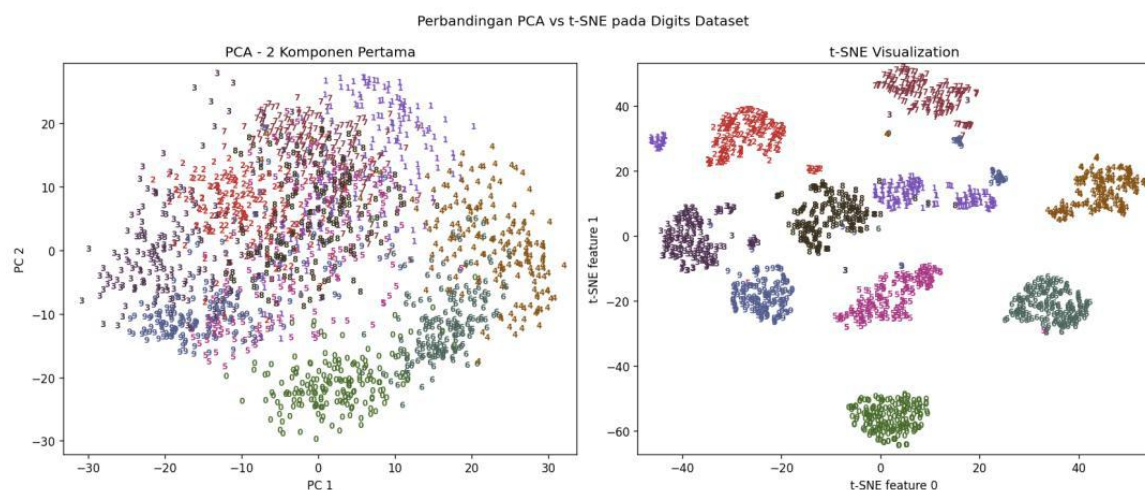
Kode 3.5: Visualisasi Data Berdimensi Tinggi dengan t-SNE Manifold

```
from sklearn.manifold import TSNE
from sklearn.datasets import load_digits

digits = load_digits()

# Transformasi Manifold t-SNE pada Citra Digit 64-Dimensi ke 2D
tsne = TSNE(random_state=42)
digits_tsne = tsne.fit_transform(digits.data)

print("Bentuk Data Asli:", digits.data.shape)
print("Bentuk Embedding t-SNE:", digits_tsne.shape)
```



Gambar 3.6: Perbandingan Visualisasi Reduksi Dimensi Digits Dataset: PCA vs t-SNE.

Perbandingan PCA dan t-SNE. PCA menghasilkan tumpukan proyeksi linear yang saling bertindihan antarkelas angka. t-SNE sebaliknya berhasil memisahkan kesepuluh kelas digit (0 hingga 9) ke dalam pulau-pulau klaster yang terisolasi tajam. Keunggulan visual itu ada ongkosnya. Jarak antarpulau pada peta t-SNE tidak bermakna apa pun karena yang dijaga hanya kedekatan lokal, dan sampel yang baru datang tidak bisa diletakkan ke peta lama tanpa menjalankan ulang seluruh proses fitting.

3.5 Algoritma Pengelompokan (Clustering)

3.5.1 k-Means Clustering

k-Means membagi data ke dalam k kelompok lewat dua langkah yang diulang bergantian ala Expectation-Maximization: tiap titik ditugaskan ke centroid terdekat, lalu

tiap centroid digeser ke rata-rata anggotanya yang baru. Iterasi berhenti ketika posisi centroid tidak lagi berpindah. Pendekatan ini paling pas untuk kelompok yang menyebar membulat dan berukuran setara, persis seperti pada Blobs dataset.

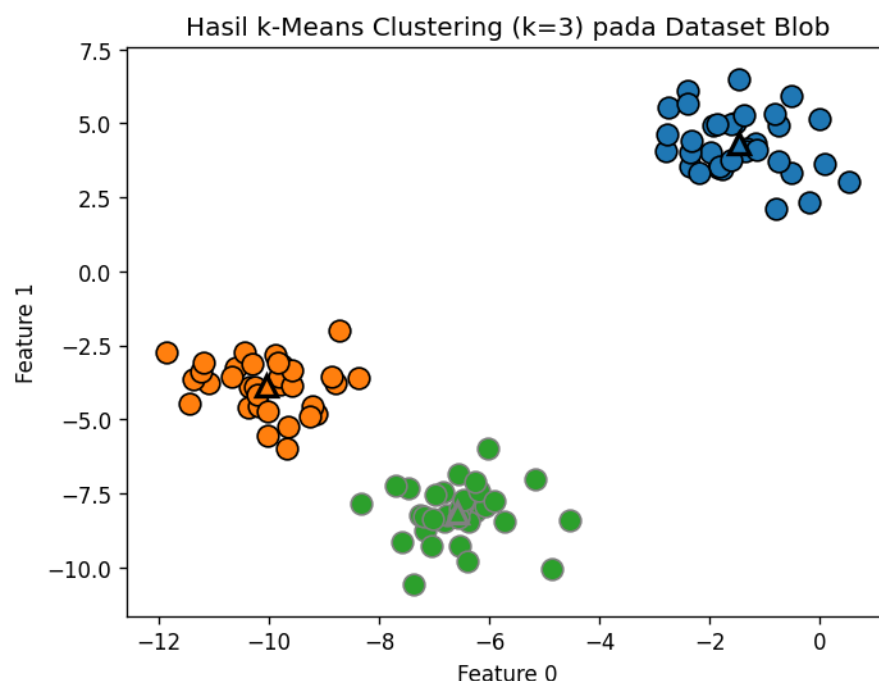
Kode 3.6: Implementasi k-Means Clustering dan Centroid Extraction

```
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

X_blobs, _ = make_blobs(random_state=1)

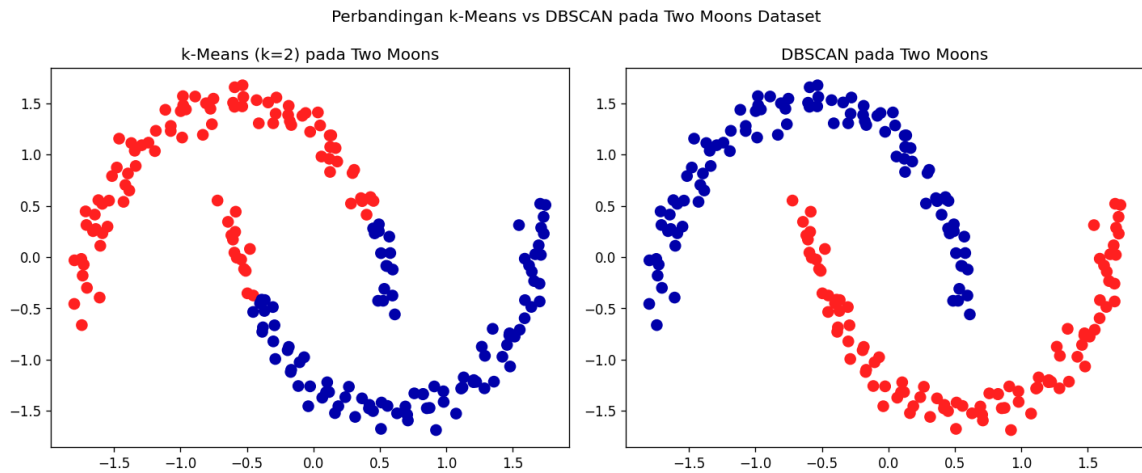
# k-Means Clustering dengan k=3
kmeans = KMeans(n_clusters=3, random_state=0)
kmeans.fit(X_blobs)

print("Posisi Cluster Centroids:\n", kmeans.cluster_centers_)
print("Distribusi Klaster Anggota:", np.bincount(kmeans.labels_))
```



Gambar 3.7: Hasil Partisi k-Means ($k=3$) dan Posisi Centroid Segitiga pada Blobs Dataset.

Keterbatasan pada struktur non-konveks. Sekat antarklaster yang dibentuk k-Means selalu berupa garis lurus (Voronoi tessellation). Begitu datanya melengkung seperti Two Moons, aturan itu langsung menjadi beban. Kedua bulan sabit terbelah diagonal, dan tiap klaster hasilnya justru memuat potongan dari dua bulan sekaligus.



Gambar 3.8: Kegagalan k-Means vs Keberhasilan Sempurna DBSCAN pada Two Moons Dataset.

3.5.2 Agglomerative Hierarchical Clustering

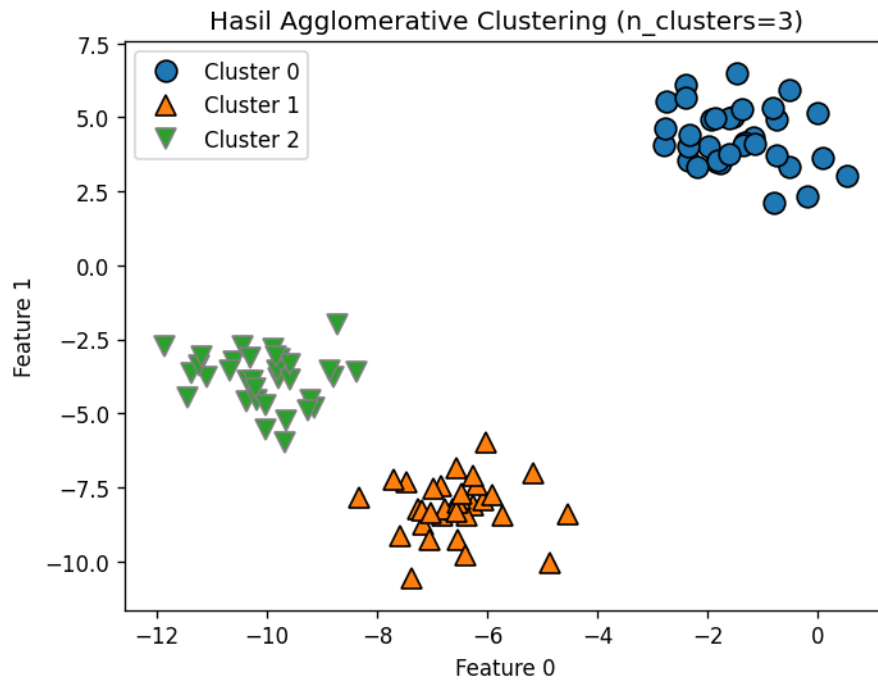
Agglomerative clustering memakai pendekatan bottom-up. Setiap sampel awalnya diperlakukan sebagai kluster mandiri, lalu dua kluster terdekat digabungkan berulang kali sampai jumlah kluster yang diinginkan tercapai. Metode Ward linkage bekerja dengan meminimalkan pertambahan varians di dalam kluster.

Kode 3.7: Hierarchical Agglomerative Clustering & Dendrogram

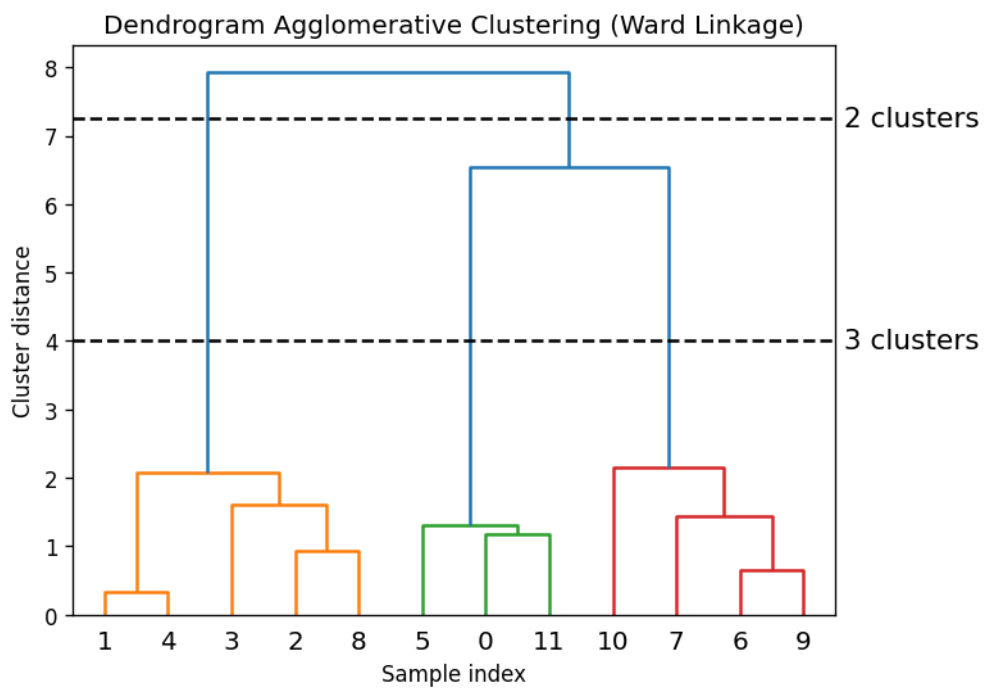
```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, ward

# Agglomerative Clustering dengan Ward Linkage
agg = AgglomerativeClustering(n_clusters=3, linkage='ward')
assignment = agg.fit_predict(X_blobs)

# Konstruksi Dendrogram Hierarki
linkage_array = ward(X_blobs[:12])
dendrogram(linkage_array)
```



Gambar 3.9: Hasil Partisi Agglomerative Clustering dengan 3 Klaster.



Gambar 3.10: Visualisasi Pohon Hierarki Dendrogram dan Pemotongan Ambang Jarak Klaster.

3.5.3 DBSCAN (Density-Based Spatial Clustering of Applications with Noise)

Berbeda dari dua metode sebelumnya, DBSCAN tidak meminta jumlah klaster ditetapkan lebih dulu. Jumlahnya muncul sendiri dari kepadatan data. Sebuah titik diangkat menjadi 'core sample' bila di dalam radius ϵ di sekelilingnya terdapat sekurang-kurangnya min_samples titik lain; klaster lalu tumbuh dengan merambat dari

satu core sample ke core sample tetangganya. Titik yang kepadatan sekitarnya tidak mencukupi ditinggalkan sebagai noise dengan label -1. Pada Two Moons, cara kerja ini membelah data menjadi tepat 2 klaster tanpa menyisakan satu pun noise point.

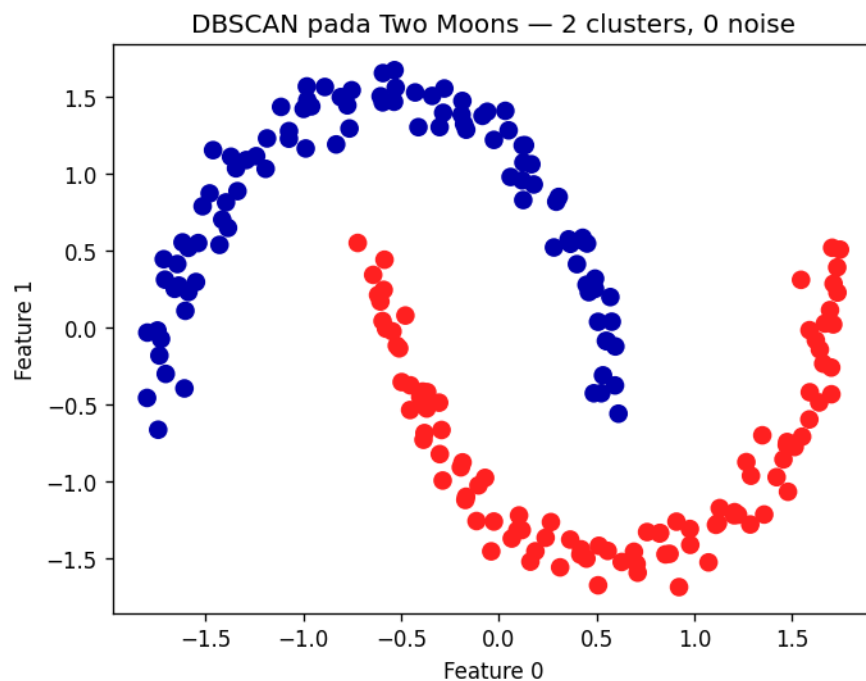
Kode 3.8: DBSCAN Spasial Berbasis Kepadatan pada Two Moons

```
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_moons

X_m, y_m = make_moons(n_samples=200, noise=0.05, random_state=0)
X_m_scaled = StandardScaler().fit_transform(X_m)

# DBSCAN: Density-Based Clustering
dbscan = DBSCAN(eps=0.2, min_samples=5)
clusters = dbscan.fit_predict(X_m_scaled)

print("Label Klaster Ditemukan:", np.unique(clusters))
print("Jumlah Titik Noise:", np.sum(clusters == -1))
```



Gambar 3.11: Hasil Segmentasi Kepadatan DBSCAN pada Two Moons.

3.6 Evaluasi Kuantitatif Clustering (ARI dan NMI)

Menilai hasil clustering tidak bisa memakai akurasi biasa. Nomor yang diberikan pada sebuah kluster sifatnya sembarang, sehingga partisi yang benar pun akan terbaca salah begitu penomorannya tertukar. Adjusted Rand Index (ARI) dan Normalized Mutual Information (NMI) menjawab persoalan itu dengan membandingkan pasangan keanggotaan, bukan label itu sendiri, sehingga skornya tetap sama berapa pun permutasi nomor klasternya.

Kode 3.9: Evaluasi Kuantitatif Clustering Menggunakan Ground Truth (ARI & NMI)

```
from sklearn.metrics.cluster import adjusted_rand_score, normalized_mutual_info_score

# Komparasi Metrik Evaluasi Kuantitatif ARI dan NMI pada Two Moons
algos = {
    'k-Means': KMeans(n_clusters=2, random_state=0),
    'Agglomerative (Ward)': AgglomerativeClustering(n_clusters=2),
    'DBSCAN': DBSCAN(eps=0.2, min_samples=5)
}

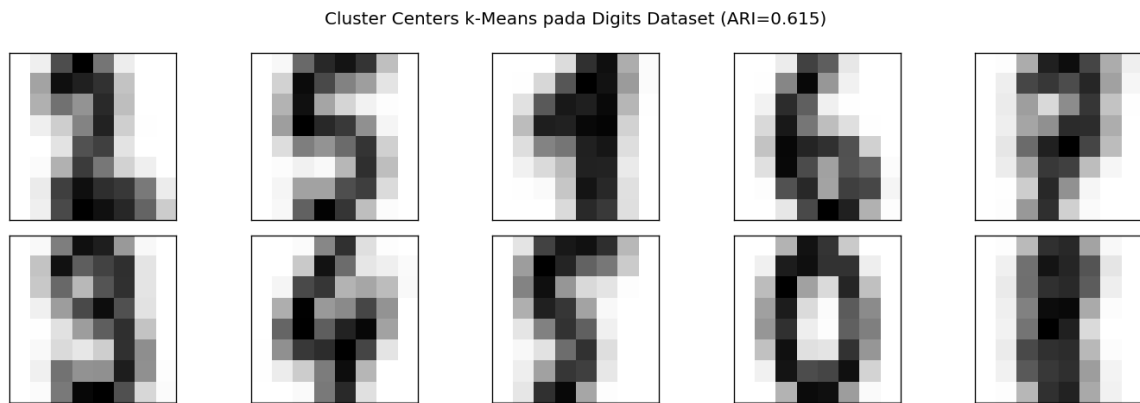
for name, algo in algos.items():
    pred = algo.fit_predict(X_m_scaled)
    ari = adjusted_rand_score(y_m, pred)
    nmi = normalized_mutual_info_score(y_m, pred)
    print(f"{name:22s} -> ARI: {ari:.4f} | NMI: {nmi:.4f}")
```

Tabel 5. Evaluasi kuantitatif algoritma clustering pada Two Moons dataset.

Algoritma Clustering	Skor ARI (Two Moons)	Skor NMI (Two Moons)	Kemampuan Menangani Bentuk Kompleks
Random Assignment (Baseline)	0.0031	0.0012	Tidak ada pemisahan struktural
k-Means Clustering	0.5016	0.3951	Buruk pada kluster non-konveks
Agglomerative (Ward)	0.6065	0.5240	Sedang, terikat pada kedekatan geometris
DBSCAN (Density-Based)	1.0000	1.0000	Sempurna (100% klasterisasi benar)

Studi Kasus Clustering pada Digits Dataset

Pengujian k-Means pada 1.797 sampel Digits menghasilkan skor ARI 0.6154 dan NMI 0.7306. Rekonstruksi visual dari titik pusat kluster (cluster centers) memperlihatkan prototipe citra rata-rata setiap angka yang berhasil dipelajari tanpa pengawasan label.



Gambar 3.12: Rekonstruksi 10 Pusat Kluster k-Means Merepresentasikan Angka 0-9.

BAB IV KESIMPULAN DAN PANDUAN PEMILIHAN MODEL

4.1 Sintesis Hasil Eksperimen

Dari seluruh rangkaian pengujian empiris pada Chapter 2 dan Chapter 3, lima kesimpulan berikut dapat ditarik:

1. **Kompromi bias-varians.** Model sederhana seperti Regresi Linier dan Decision Tree tanpa batas kedalaman berada pada kutub ekstrem underfitting atau overfitting. Teknik regulasi L1/L2 (Ridge dan Lasso) serta pre-pruning pohon terbukti wajib untuk menjamin kemampuan generalisasi.
2. **Keunggulan ensemble.** Random Forest dan Gradient Boosted Trees secara konsisten mendominasi performa klasifikasi tabular dengan akurasi 97.20%, sekaligus mengatasi kelemahan instabilitas pohon tunggal.
3. **Peran penskalaan fitur.** Model berbasis jarak dan margin (SVM, KNN, MLP) mutlak membutuhkan normalisasi data melalui MinMaxScaler atau StandardScaler. Penskalaan berhasil mengangkat akurasi pengujian SVM dari 94.41% menjadi 96.50%.
4. **Komplementaritas reduksi dimensi.** PCA unggul untuk kompresi data dan deteksi fitur dominan melalui rekonstruksi varians, sementara t-SNE tak tertandingi untuk inspeksi visual kluster non-linear.
5. **Topologi clustering.** Pilihan algoritma pengelompokan bergantung pada bentuk sebaran datanya. Selama klasternya membulat dan berukuran setara, k-Means sudah memadai. Begitu konturnya tidak beraturan atau ada penciran yang harus disisihkan, DBSCAN jauh lebih tepat karena bertumpu pada kepadatan, bukan pada jarak ke sebuah titik pusat.

4.2 Panduan Keputusan Praktis (Algorithm Selection Guide)

Sebagai rujukan operasional dalam proyek machine learning di masa depan, Tabel 6 menyajikan matriks rekomendasi pemilihan algoritma.

Tabel 6. Matriks rekomendasi pemilihan algoritma machine learning.

Kasus Masalah / Tipe Data	Algoritma Rekomendasi Utama	Alternatif Cepat / Baseline	Faktor Kunci Keberhasilan
Klasifikasi Tabular Sedang-Besar	Random Forest / GBRT	Logistic Regression	Tuning n_estimators & max_depth

Kasus Masalah / Tipe Data	Algoritma Rekomendasi Utama	Alternatif Cepat / Baseline	Faktor Kunci Keberhasilan
Data Berdimensi Tinggi (Sparse / Teks)	Linear SVM / Logistic Reg.	Naive Bayes (Multinomial)	Regulasi L1/L2, penskalaan fitur
Regresi Prediksi Numerik Kontinu	Ridge Regression / GBRT	Linear Regression	Tuning parameter penalti alpha
Visualisasi Data Kompleks (>3 Dimensi)	t-SNE (Manifold)	PCA (2 Komponen)	Tuning perplexity, jangan gunakan untuk fitur baru
Kompresi Data / Denoising Fitur	PCA / NMF	Feature Selection	Evaluasi rasio varians kumulatif (>85%)
Clustering Bentuk Bola Sederhana	k-Means Clustering	MiniBatchKMeans	Tentukan k dengan Elbow / Silhouette method
Clustering Spasial dengan Outlier	DBSCAN	Agglomerative Clustering	Tuning parameter radius eps & min_samples